ER Model Part 2 – Terminology Concepts Reference

Database Systems

Based on Lecture 6 by Tiantian Liu

Fall 2025

Contents

1 Quick Reference Guide

ER to Relational Mapping – At a Glance

Seven-Step Mapping Process:

- **M1**: Regular entity types → Create relation
- **M2**: Weak entity types → Create relation with foreign key
- **M3**: Binary M:N relationships → Create new relation
- **M4**: Binary 1:N relationships → Extend N-side relation
- **M5**: Binary 1:1 relationships → Extend one relation
- **M6**: Complex relationships → Handle recursion/n-ary
- **M7**: Complex attributes → Handle multi-valued/composite/derived

Key Principles:

- Primary keys become underlined attributes
- Foreign keys reference other relations (notation: FK–>Relation)
- Minimize NULL values by choosing appropriate mapping strategy
- Avoid redundancy while preserving all information

2 Data Modelling Process Review

From ER Diagram to Relational Tables

The mapping process converts a DBMS-independent conceptual schema (ER diagram) into a DBMS-specific logical schema (relational tables).
Steps in Database Design:

1. **Requirement Collection and Analysis** – Gather requirements
2. **Conceptual Design** – Create ER diagram
3. **Data Model Mapping** – Convert ER to relational schema (*This lecture!*)
4. **Physical Design** – Implement in specific DBMS

Why is mapping important?

- ER diagrams are conceptual and cannot be directly implemented
- Relational databases need tables, not entity-relationship diagrams
- Mapping rules ensure no information is lost in the translation

3 M1: Regular Entity Types

Mapping Regular Entity Types

Rule: Create a relation for each regular (strong) entity type.
Procedure:

1. Create a new relation with the same name as the entity type
2. Add a column for each simple attribute
3. The primary key of the entity becomes the primary key of the relation
4. Composite attributes are flattened (handled in M7)

Special Case: If an entity has no attributes other than the primary key and participates totally in a relationship, the relation can be eliminated and merged with the relationship relation.

M1 Examples

Example 1 – Course Entity:
ER: `course(ID, title, credit)`
Relation: `course(ID, title, credit)`
Example 2 – Instructor Entity:
ER: `instructor(ID, name, salary)`
Relation: `instructor(ID, name, salary)`
Example 3 – Entity with No Extra Attributes:
If `meeting(number)` has total participation in a relationship and no other attributes, it can be eliminated and its key can be included in the relationship relation.

4 M2: Weak Entity Types

Mapping Weak Entity Types

Rule: Create a relation for each weak entity type.
Procedure:

1. Create a new relation with the entity type name
2. Add columns for all simple attributes
3. Add a foreign key referencing the strong (identifying) entity
4. Primary key = foreign key + discriminator (partial key)

Important Notes:

- Weak entity and identifying relationship are always merged
- If weak entity relates to strong entity via 1:1, it can be merged with strong entity

M2 Examples

Example 1 – Section (Weak Entity):
ER:

- `course(course_id, title, credit)` – strong entity
- `section(sec_id, semester, year)` – weak entity

Relations:

- `course(course_id, title, credit)`
- `section(course_id->course, sec_id, semester, year)`

Primary key of section includes `course_id` (from strong entity) and discriminator (`sec_id`, `semester`, `year`).
Example 2 – Copy (Weak Entity):
ER:

- `book(ISBN, title, author)` – strong entity
- `copy(CID, version)` – weak entity

Relations:

- `book(ISBN, title, author)`
- `copy(ISBN->book, CID, version)`

Weak Entity Constraints

Remember:

- Weak entities MUST have total participation in identifying relationship
- Identifying relationship must be 1:N or 1:1 (never M:N)
- Strong entity is always on the "1" side

5 M3: Binary M:N Relationship Types

Mapping M:N Relationships

Rule: Create a new relation for each binary M:N relationship type.
Procedure:

1. Create a new relation with the relationship name
2. Add foreign keys referencing the primary keys of both participating entities
3. Primary key = union of both foreign keys
4. Add columns for relationship attributes (if any)

Why a new relation? M:N relationships cannot be represented by simply adding a foreign key to one side, as both sides can have multiple associations.

M3 Examples

Example 1 – Supervise (M:N):
ER:

- `instructor(i_id, name, salary)`
- `student(s_id, name)`
- `supervise(meeting)` – relationship attribute

Relations:

- `instructor(i_id, name, salary)`
- `student(s_id, name)`
- `supervise(i_id->instructor, s_id->student, meeting)`

The primary key is (`i_id`, `s_id`) because both are needed to uniquely identify a supervision relationship.
Example 2 – Borrows (M:N):
ER:

- `member(ID, name, phone)`
- `book(ISBN, title, author)`
- `borrows(date)` – relationship attribute

Relations:

- `member(ID, name, phone)`
- `book(ISBN, title, author)`
- `borrows(ID->member, ISBN->book, date)`

6 M4: Binary 1:N Relationship Types

Mapping 1:N Relationships

Rule: Extend the N-side relation with a foreign key.
Two Options:
Option 1: Create a separate relation (not recommended)

- Primary key is the N-side entity's primary key

Option 2: Extend the N-side table (RECOMMENDED)

- Add foreign key from 1-side to N-side relation
- Add relationship attributes to N-side relation
- This avoids creating an extra table

Why extend N-side?

- Extending 1-side would cause redundancy (many NULL values)
- Each entity on N-side relates to at most one on 1-side
- Update, deletion, and insertion anomalies avoided

M4 Examples

Example 1 – Supervise (1:N):
ER: One instructor supervises many students
Relations (Option 2 - Recommended):

- `instructor(i_id, name, salary)`
- `student(s_id, name, i_id->instructor, meeting)`

The foreign key `i_id` is added to the student table (N-side).
Example 2 – Holds (1:N):
ER: One branch holds many books
Relations:

- `branch(ID, name, location)`
- `book(ISBN, title, author, branchID->branch)`

Why not 1-side?
If we extended the instructor table instead:

- Many instructors don't supervise students → many NULL values
- If one instructor supervises multiple students, we'd have duplicate rows
- This violates normalization principles

Common Mistakes in M4

Don't extend the 1-side!
Extending the 1-side causes:

- **Update anomaly**: If Einstein supervises two students, his data appears twice
- **Deletion anomaly**: Removing a student might remove instructor data
- **Insertion anomaly**: Cannot add new instructors who don't supervise students

Always extend the N-side to avoid these problems.

7 M5: Binary 1:1 Relationship Types

Mapping 1:1 Relationships

Rule: Extend one of the relations with a foreign key.
Strategy:

- **Best choice**: Extend the relation with total participation (avoids NULLs)
- **If both partial**: Choose either side
- **If both total**: Can merge into one relation (eliminates JOIN operations)

Three Cases:

1. **One total, one partial**: Extend total participation side
2. **Both partial**: Extend either (prefer side with fewer NULLs)
3. **Both total**: Merge into single relation OR extend either side

M5 Examples

Example 1 – Supervise (1:1, Student Total):
ER:

- Every student must have exactly one supervisor
- Instructors may or may not supervise

Relations (Extend student - total participation):

- `instructor(i_id, name, salary)`
- `student(s_id, name, i_id->instructor, meeting)`

Example 2 – Has (1:1, Both Total):
ER: Person and Passport (every person has passport, every passport has person)
Option 1 (Extend either):

- `person(ID, name, phone)`
- `passport(pID, country, expiry_date, ID->person, issued_number)`

Option 2 (Merge):

- `r(ID, name, phone, pID, country, expiry_date, issued_number)`

Example 3 – Both Partial:
If both instructor and student have partial participation:

- Extending either side works
- Choose based on which side has fewer NULL values expected

Why Not the Partial Side?

Example: If we extended instructor (partial) instead:

- Many instructors don't supervise → `s_id` and `meeting` would be NULL
- Wastes storage space
- Complicates queries with NULL checks

Extending the total participation side ensures no NULLs!

8 M6: Complex Relationship Types

8.1 Recursive Relationships

Mapping Recursive Relationships

Rule: Map like standard relationships, but rename foreign keys using role names.
For M:N recursive:

- Create new relation with two foreign keys to same entity
- Use role names to distinguish (e.g., `after-course`, `pre-course`)

For 1:N recursive:

- Add foreign key to same entity relation
- Use role name to distinguish

Recursive Relationship Examples

Example 1 – Course Prerequisites (M:N):
ER: Courses have prerequisites that are also courses
Relations:

- `course(course_id, title, credit)`
- `prereq(after-course_id->course, pre-course_id->course)`

Example 2 – Course Prerequisites (1:N):
ER: Each course has at most one prerequisite
Relations:

- `course(course_id, title, credit, pre-course_id->course)`

Example 3 – Employee Supervision:
ER: Employees supervise other employees (1:N)
Relations:

- `employee(e_id, name, supervisor_id->employee)`

8.2 N-ary Relationships

Mapping N-ary Relationships

For N:M:P (all many):

- Create new relation
- Primary key = union of all participating entity keys
- Add foreign keys for all entities

For N:M:1 (one has "1"):

- Create new relation
- Primary key = keys of entities with "M" and "N" only
- Add foreign keys for all entities

N-ary Relationship Examples

Example 1 – Project Guidance (N:M:P):
ER: Students work on projects under instructor guidance (M:N:P)
Relations:

- `student(s_id, name)`
- `project(p_id, title)`
- `instructor(i_id, name)`
- `proj_guide(s_id->student, p_id->project, i_id->instructor)`

Example 2 – Transport (N:M:1):
ER: Drivers transport vehicles, one driver per transport (N:M:1)
Relations:

- `driver(d_id, name)`
- `vehicle(v_id, model)`
- `transport(d_id->driver, departure, v_id->vehicle, arrival)`

Primary key is (`d_id`, `departure`) because driver + departure uniquely identifies transport. The vehicle side has "1" so it's not part of the primary key.

9 M7: Complex Attributes

9.1 Multi-valued Attributes

Mapping Multi-valued Attributes

Rule: Create a separate relation for each multi-valued attribute.
Procedure:

1. Create new relation named after the attribute
2. Add foreign key referencing the original entity
3. Add column for the attribute value
4. Primary key = foreign key + attribute value

Multi-valued Attribute Examples

Example – Employee with Multiple Phone Numbers:
ER: `employee(e_id, name, salary, phone)`
Where `phone` is multi-valued (double oval in ER diagram).
Relations:

- `employee(e_id, name, salary)`
- `phone(e_id->employee, phoneNumber)`

Each phone number becomes a separate row in the `phone` table.

9.2 Composite Attributes

Mapping Composite Attributes

Rule: Include only the component (leaf) attributes in the relation.
Procedure:

- Flatten the composite attribute
- Create columns for each simple component
- Do not create a column for the composite attribute itself

Composite Attribute Examples

Example 1 – Employee with Address:
ER: `employee(e_id, name, salary, address)`
Where `address` is composite: `address(street, city, state)`
Relation:

- `employee(e_id, name, salary, street, city, state)`

Example 2 – Employee with Name:
ER: `employee(e_id, name, salary)`
Where `name` is composite: `name(first, middle, last)`
Relation:

- `employee(e_id, first, middle, last, salary)`

9.3 Derived Attributes

Mapping Derived Attributes

Rule: Ignore derived attributes during mapping.
Rationale:

- Derived attributes can be computed from other attributes
- Storing them causes redundancy
- They can be added later using database views

Example:

- `age` can be derived from `birth_date`
- `total_price` can be derived from `quantity` × `unit_price`

Derived Attribute Example

Example – Employee Age:
ER: `employee(e_id, name, birth_date, age)`
Where `age` is derived (dashed oval in ER diagram).
Relation:

- `employee(e_id, name, birth_date)`

The `age` attribute is not stored. Instead, it can be computed:

```
SELECT e_id, name, YEAR(CURRENT_DATE) - YEAR(birth_date) AS age FROM
employee;
```

10 Mapping Summary

Complete Mapping Overview

M1: Regular Entity Types

- Create relation with all simple attributes
- Primary key = entity's primary key

M2: Weak Entity Types

- Create relation with foreign key to strong entity
- Primary key = strong entity key + discriminator

M3: Binary M:N Relationships

- Create new relation with both foreign keys
- Primary key = union of foreign keys

M4: Binary 1:N Relationships

- Extend N-side relation with foreign key from 1-side
- Add relationship attributes to N-side

M5: Binary 1:1 Relationships

- Extend relation with total participation (if any)
- If both total: merge into one relation or extend either
- If both partial: extend either side

M6: Complex Relationships

- Recursive: Create relation with renamed foreign keys (roles)
- N-ary: Create relation with all foreign keys; PK depends on cardinalities

M7: Complex Attributes

- Multi-valued: Create separate relation
- Composite: Include only component attributes
- Derived: Ignore (can use views)

11 Specialization/Generalization Mapping

Relational Modeling of Specialization

The relational model does not directly support inheritance (ISA relationships). Four alternatives exist for mapping specialization hierarchies:

Alternative 1: Main Classes

- Create separate relations for each entity class
- Each relation has all inherited + specific attributes
- Entities appear only in their most specific relation

Alternative 2: Partitioning

- Create separate relations for super and subclasses
- Subclass relations contain only specific attributes + foreign key
- Primary key is duplicated across relations

Alternative 3: Full Redundancy

- Create separate relations for each class
- All attributes (inherited + specific) stored in each relation
- Superclass relation contains all entities

Alternative 4: Single Relation

- Store all entities in one relation
- Add type attribute to indicate entity class
- Many NULL values for subclass-specific attributes

11.1 Alternative 1: Main Classes

Alternative 1 Example

ER Hierarchy:

- `person(ID, name, gender)`
- `employee` ISA `person` (salary)
- `student` ISA `person` (credits)

Relations:

- `person(ID, name, gender)` – only for persons who are neither employee nor student
- `employee(ID, name, gender, salary)` – all inherited + specific
- `student(ID, name, gender, credits)` – all inherited + specific

Characteristics:

- **Pros**: Simple queries for specific types; no joins needed
- **Cons**: Data duplication if entity belongs to multiple subclasses; hard to query all persons

11.2 Alternative 2: Partitioning

Alternative 2 Example

Relations:

- `person(ID, name, gender)` – all persons
- `employee(ID->person, salary)` – only specific attributes
- `student(ID->person, credits)` – only specific attributes

Characteristics:

- **Pros**: No redundancy; easy to query all persons; supports overlapping specialization
- **Cons**: Requires JOIN for subclass queries; slightly more complex

Note: This is the most normalized approach and generally preferred.

11.3 Alternative 3: Full Redundancy

Alternative 3 Example

Relations:

- `person(ID, name, gender)` – all persons
- `employee(ID, name, gender, salary)` – employees with inherited attributes
- `student(ID, name, gender, credits)` – students with inherited attributes

Characteristics:

- **Pros**: Simple queries for specific types; no joins needed
- **Cons**: Significant data redundancy; update anomalies if name/gender changes

Use Case: Read-heavy applications where JOIN performance is critical.

11.4 Alternative 4: Single Relation

Alternative 4 Example

Relation:

- `person(ID, name, gender, type, salary, credits)`

Where `type` $\in$ {person, employee, student}
Sample Data:

ID	name	gender	type	salary	credits
10001	Alice	F	student	NULL	100
12121	Wu	M	employee	90000	NULL
12930	Sophia	F	person	NULL	NULL

Characteristics:

- **Pros**: Single table; simple schema; easy to query all persons
- **Cons**: Many NULL values; wastes storage; integrity harder to enforce

Use Case: Small hierarchies with few subclass-specific attributes.

12 Practical Example: Car Insurance Company

Complete Mapping Example

Requirements:

- Customers own one or more cars
- Cars have zero or more recorded accidents
- Insurance policies cover one or more cars
- Policies have one or more payments

- Payments have due date and received date

Relations:

- `customer(customerID, address, name)`
- `car(vehicleIDnumber, model, licenseNo, customerID->customer, policyID->policy)`
- `policy(policyID)`
- `payment(policyID->policy, paymentNo, receivedOn, amount, dueDate)`
- `accident(reportID, date, place)`
- `participated(vehicleIDnumber->car, reportID->accident)`

Mapping Decisions:

- **owns** (1:N): Foreign key `customerID` in `car` table
- **covers** (1:N): Foreign key `policyID` in `car` table
- **participated** (M:N): Separate relation with composite key
- **paying** (1:N): `payment` is weak entity with `policyID` as foreign key

13 Best Practices and Common Mistakes

Common Mapping Mistakes

1. **Wrong side for 1:N foreign key:**
   - Always add foreign key to N-side
   - Adding to 1-side causes redundancy and anomalies
2. **Missing foreign keys in weak entities:**
   - Weak entity primary key MUST include strong entity's key
   - Don't forget to mark it as a foreign key
3. **Not creating relation for M:N:**
   - M:N relationships ALWAYS need a separate relation
   - Cannot be represented by foreign key alone
4. **Storing derived attributes:**
   - Don't store values that can be calculated
   - Use views instead
5. **Incorrect primary keys:**
   - M:N relation: PK = union of both foreign keys
   - 1:N relation: PK = N-side's original key
   - Weak entity: PK = strong entity key + discriminator

Design Tips

Minimize NULL values:

- For 1:1, extend side with total participation
- For specialization, use partitioning approach

Choose appropriate specialization mapping:

- **Partitioning**: Most normalized; best for overlapping specialization
- **Single relation**: Simple but many NULLs; use for small hierarchies
- **Main classes**: Good for disjoint specialization with rare superclass queries
- **Full redundancy**: Use only for read-heavy, performance-critical applications

Naming conventions:

- Use descriptive names for relations and attributes
- Foreign keys often named after referenced entity + "ID"
- Use role names for recursive relationships

Documentation:

- Clearly mark primary keys (underline)
- Clearly indicate foreign keys (FK–>Relation)
- Document any mapping choices that deviate from standard rules

14 Summary

Key Takeaways

Mapping Process:

1. Start with regular entities (M1)
2. Handle weak entities (M2)
3. Map M:N relationships first (M3)
4. Then 1:N relationships (M4)
5. Then 1:1 relationships (M5)
6. Handle complex cases (M6)
7. Finally, complex attributes (M7)

Critical Rules:

- Every entity type becomes a relation (except in special cases)
- M:N relationships ALWAYS need a separate relation
- 1:N relationships: extend N-side
- 1:1 relationships: extend side with total participation
- Weak entities: include strong entity's key in primary key
- Multi-valued attributes: create separate relation
- Derived attributes: ignore (use views)

Quality Metrics:

- Minimize NULL values
- Avoid redundancy
- Preserve all information from ER diagram
- Ensure all constraints are enforceable